

Conway's Game of Life
Irene Hong
Computer Architecture | Fall 2025

Overview

I designed a digital circuit that plays three of Conway's Games of Life displayed under separate RGB channels using a 8x8 WS2812B RGB LED, an iCE40UP5K FPGA, and the Nitro FPGA bootloader. The game is played infinitely on the LED matrix with cyclic boundary conditions. For instance, cells in row 0 consider row 7 as their previous row and cells in row 7 consider row 0 as their next. The same logic applies to columns. The RGB color space represents the cell states as living (colored) and dead (no color). If any of these cells overlap, the corresponding channels create a new color (ie. magenta or cyan).

Circuit Architecture

Controller

The entire system is run under a state machine of TRANSMIT_FRAME cycling through all 64 pixels on the grid and IDLE waiting for 1,048,574 cycles after transmitting a new frame to ensure a visible frame rate. orchestrated by the controller module. TRANSMIT_FRAME has three phases: reading channel values from 3 game of life instances, loading the 24-bit RGB data into the shift register, and transmitting the pixel data to the LEDs over 360 cycles (15 cycles per bit).

Game of Life

Three game of life instances run independently (one for each color channel). The module has two local memories: read_memory (an 8x8 2D array representing the current frame) and write_memory (a 64-length 1D array representing the next frame). The initial memory is loaded from the config files (which lay out the initial patterns explained above). These memories store 64 bits of 0 or 1—0 for dead and 1 for alive. For each pixel that is being evaluated in the game of life instance, the number of alive neighbors are counted out of the 8 adjacent pixels, and Conway's rules of life are applied to determine the pixel's new state (and therefore, its new value of 0 or 1).

This game of life logic is computed using sequential logic under the negative edge of the clock. New pixel values are then updated to the corresponding index in write_memory. There is also a write_flag that indicates when the read_memory should be replaced by the write_memory. This occurs when all new 64 pixel values have been inputted into write_memory. All changes to the local memory occur on the positive edge of the clock. This edge separation prevents race conditions and ensures clean data flow.

WS2812b

The WS2812B driver module generates the timing-critical protocol for controlling RGB LEDs. Each bit is transmitted over 15 clock cycles: logic 0 is high for 5 cycles and low for 10 cycles, while logic 1 is high for 10 cycles and low for 5 cycles. The module samples the serial input at the start of each bit transmission and generates the corresponding waveform.

Top

The top module integrates all components. It maintains a 24-bit shift register holding RGB data in GRB format (8 bits per channel). Each color channel's single-bit state is expanded to 8 bits with the repetition operator to represent their brightness value from 0-255.

Design Decisions

Memory Indexing

The 64-pixel address space maps to the 8×8 grid through bit extraction: row = address[5:3] (upper 3 bits), column = address[2:0] (lower 3 bits). This allows sequential addressing by simply incrementing a 6-bit counter.

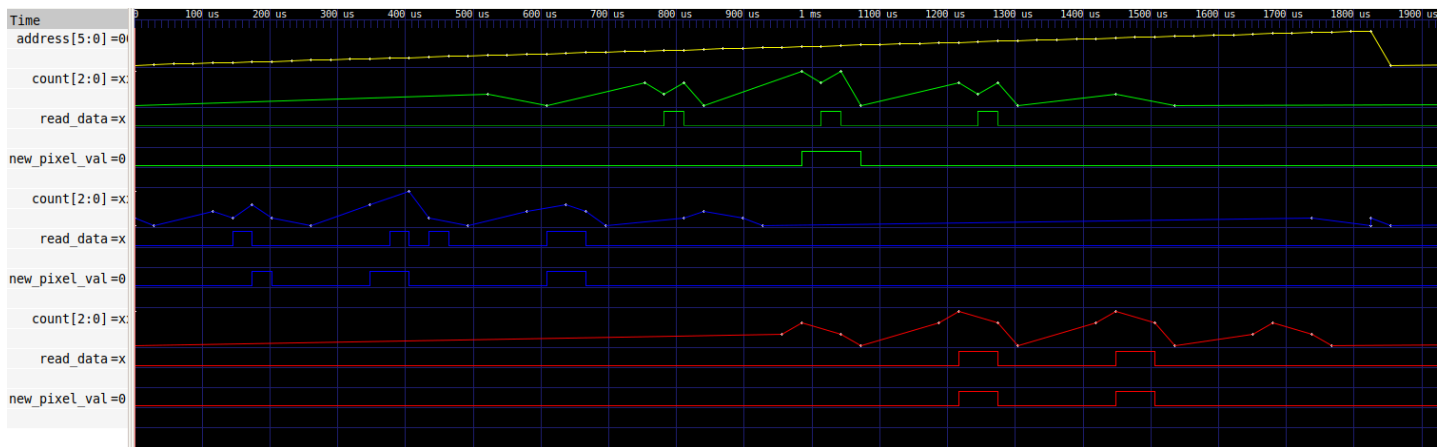
Timing Parameters

The system operates at 12 MHz clock frequency. The effective frame rate is ~11 FPS.

Initial Configurations

Three pattern files initialize the simulations: oscillator.txt contains a vertical blinker (period-2 oscillator), spaceship.txt contains a glider (moving pattern), and still_life.txt contains a 2×2 block (stable pattern). These demonstrate different classes of Game of Life behavior: oscillation, translation, and stability.

Testbench & Simulation



address is the pixel index (from 1-64, which is read from left to right, top to bottom across the matrix). count is the number of alive neighbors by the pixel that is currently being evaluated and displayed on the current frame. The value of this pixel is given by read_data. new_pixel_val is the new value of the given pixel that will be displayed in the next frame.

The simulation results effectively validate the correct operation of the Game of Life logic by showing that the value of new_pixel_val is a direct, rule-based function of the inputs: the read_data and count. The continuous, sequential sweep of the address signal confirms the system is correctly iterating through all 64 pixels to process a new frame. Importantly, the occurrence of distinct high pulses in the new_pixel_val

for the Blue (e.g., around 400 us and 600 us) and Green (around 1 ms) channels indicates successful birth or survival events, meaning the logic correctly evaluated the neighbor count (count=3 for birth, or count=2 or 3 for survival) and current state to output a 1. Conversely, the lack of births in the Red channel, despite changing counts, confirms the logic correctly outputs a 0 when the rules for survival or birth are not met. This makes sense given the pattern should be a still life. The distinct and asynchronous nature of these calculated outputs across the RGB traces further demonstrates that the three color channels are running independent and correct Game of Life computations as required.

Demo Video

<https://youtube.com/shorts/iYlejEyvorM?si=I1nh2DtT7Ljr7PPT>

Source Code

https://github.com/eneri4/iceBlinkPico/tree/main/examples/game_of_life